Firjan SENAI

Específico Profissional 2

Desenvolvimento de Sistemas

• LINGUAGENS DE PROGRAMAÇÃO • APIs E FRAMEWORKS
•INTEGRAÇÃO DE SISTEMAS • CONTROLE DE VERSÃO



2026



Sumário

1. **Fundamentos do Desenvolvimento de Sistemas**
   Visão sistêmica, planejamento, organização do trabalho e metodologias de desenvolvimento
2. **Qualidade e Requisitos do Sistema**
   Qualidade, integridade, usabilidade, segurança da informação e requisitos não funcionais
3. **Linguagens de Programação**
   Ferramentas, boas práticas e desenvolvimento multiplataforma
4. **APIs, Bibliotecas e Frameworks**
   Construção de rotinas de software e integração de recursos
5. **Integração de Sistemas**
   Comunicação entre subsistemas, plataformas e serviços
6. **Padrões de Projeto**
   Design Patterns, arquitetura e organização do código
7. **Controle de Versão**
   Gerência de configuração, repositórios, versionamento e rastreabilidade
8. **Documentação de Sistemas**
   Documentação técnica, registro das atividades e organização das informações



10

Validação de dados

Aula 37 a 40 - Desenvolvimento de Sistemas
Instrutor: Vitor Hugo Almeida de Sousa

Firjan SENAI

Validação de dados

O que acontece quando o cliente envia informações inadequadas?

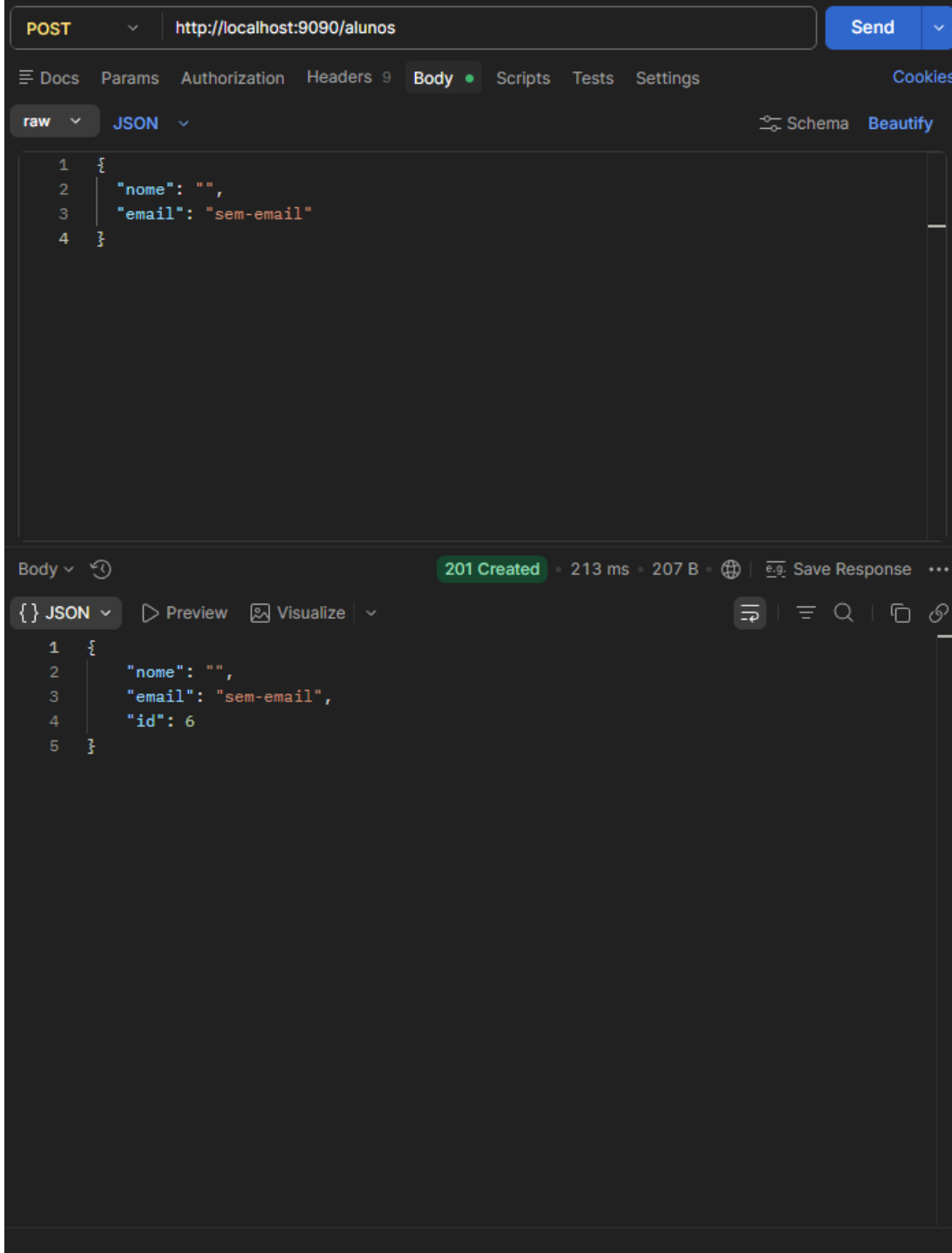
A API de alunos já recebe um JSON e grava os dados no banco. Porém, antes do cadastro, precisamos verificar se as informações recebidas atendem às regras da aplicação.

No Postman, envie esta requisição para a rota que já construímos:

- POST http://localhost:8080/alunos

Na aba Body, selecione raw → JSON e envie:

```
{
  "nome": "",
  "email": "sem-email"
}
```



Validação

O JSON tem a estrutura esperada pela API, mas os valores apresentam problemas: o nome está vazio e o e-mail não possui um formato válido.

Observe a resposta no Postman e consulte a tabela aluno. Neste momento, o cadastro pode ser aceito, pois ainda não definimos regras para verificar esses campos antes de chamar o Service.

Atenção



Receber um JSON válido não significa receber dados adequados. A validação verifica os valores enviados pelo cliente e impede que informações que não atendem às regras da aplicação sejam gravadas no banco.

Firjan SENAI

Validação no Model

Quais dados a API deve aceitar?

Vamos definir duas regras para o cadastro: o nome deve ser preenchido e o e-mail deve ser preenchido em um formato válido. Como a dependência **Validation** foi selecionada na criação do projeto, podemos usar suas anotações na classe Aluno.

No arquivo Aluno.java:

- adicione as importações
- acrescente as anotações aos atributos existentes

@NotBlank rejeita um texto vazio ou formado apenas por espaços. @Email verifica se o valor informado tem formato de e-mail. O texto definido em message descreve o problema quando a regra não é atendida.

As duas anotações aparecem em email porque são verificações diferentes: o campo deve estar preenchido e ter um formato válido.

Aluno.java

src > main > java > br > com > senai > teste > model > Aluno.java > Aluno

```java
package br.com.senai.teste.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

@Entity
@Table(name = "aluno")
public class Aluno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @NotBlank(message = "O nome é obrigatório")
    private String nome;

    @NotBlank(message = "O e-mail é obrigatório")
    @Email(message = "Informe um e-mail válido")
    private String email;

    public Aluno() {
    }

    public Aluno(String nome, String email) {
        this.nome = nome;
        this.email = email;
    }

    public int getId() {
        return id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

Firjan SENAI

Validação no Controller

Verificando o JSON antes do cadastro

Vamos definir duas regras para o cadastro: o nome deve ser preenchido e o e-mail deve ser preenchido em um formato válido. Como a dependência **Validation** foi selecionada na criação do projeto, podemos usar suas anotações na classe Aluno.

No arquivo AlunoController.java:

- adicione as importações
- No método cadastrar(), coloque @Valid antes de @RequestBody

@RequestBody converte o JSON recebido em um objeto Aluno. Em seguida, @Valid solicita a verificação das regras definidas nessa classe.

Se os dados forem válidos, o método continua e chama o Service. Se alguma regra falhar, o Spring interrompe essa requisição antes do cadastro e responde com 400 Bad Request.



Validação na atualização

Assim como no cadastrar, a atualização também ocorre envio de dados. Logo, é necessário que seja feita a validação no método atualizar também.

Fique ON



Aplique @Valid também ao parâmetro @RequestBody Aluno novosDados do método atualizar(). Assim, tanto o cadastro quanto a atualização completa verificam nome e e-mail antes de gravar os dados.

Postman

Comparando dados válidos e inválidos

Reinicie a aplicação para aplicar as alterações.

No Postman, faça duas requisições POST para:

- http://localhost:8080/alunos

Em ambas, selecione **Body** → **raw** → **JSON**:

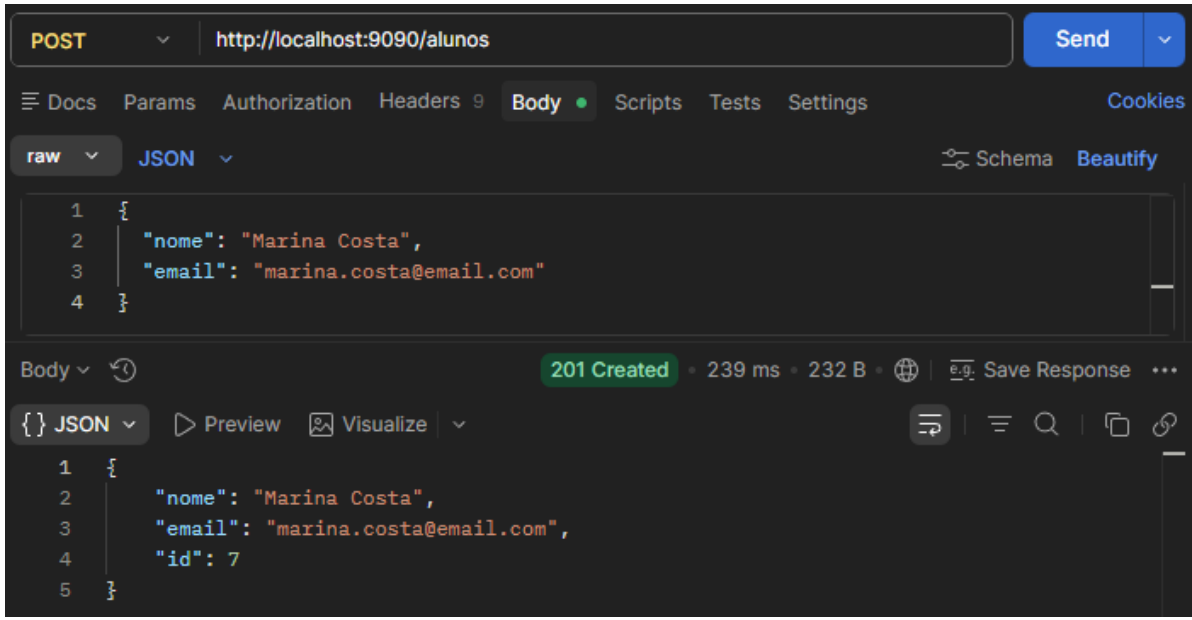
- Teste 1 — Dados válidos

  Resultado esperado: 201 Created. A resposta deverá apresentar o aluno com o ID gerado pelo banco.

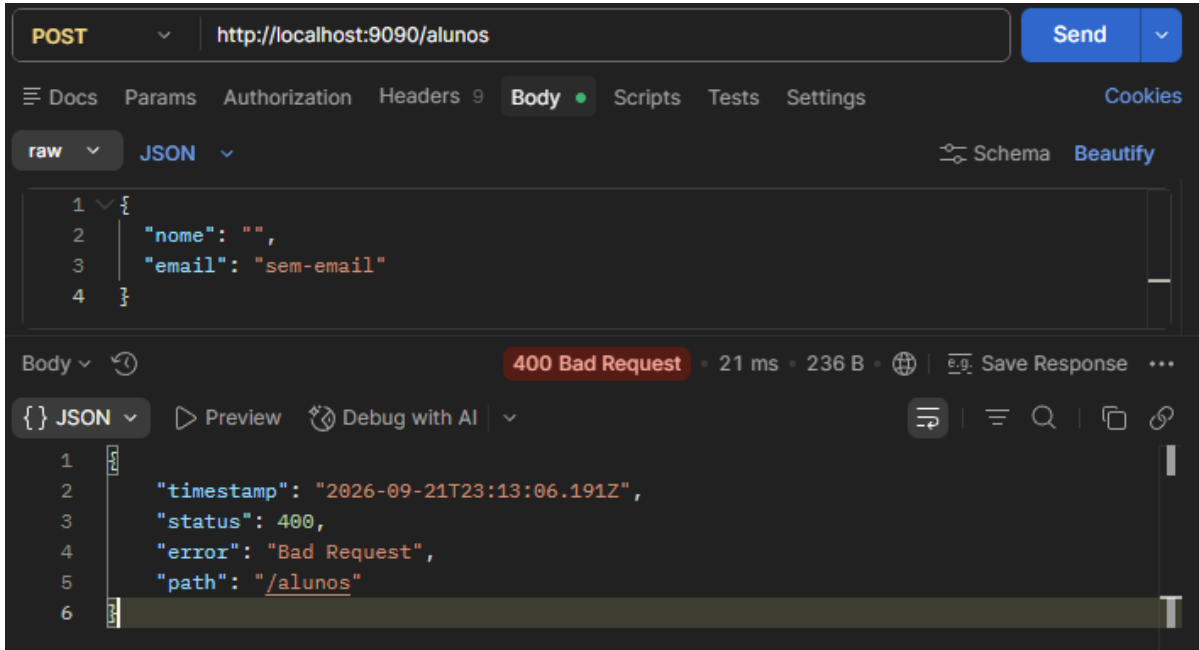
- Teste 2 — Dados inválidos

  Resultado esperado: 400 Bad Request. O aluno não deverá ser cadastrado. Consulte a tabela aluno para confirmar.

Teste 1



Teste 2



Firjan SENAI

Resposta para dados inválidos

Mostrando ao cliente qual campo precisa ser corrigido

O código 400 Bad Request informa que os dados enviados foram rejeitados. Para ajudar quem utiliza a API, vamos devolver também as mensagens que escrevemos nas anotações do Model.

Crie o pacote exception dentro do pacote principal do projeto.

Nele, crie o arquivo:

- ValidacaoExceptionHandler.java

Quando @Valid encontra dados inválidos, o Spring produz uma MethodArgumentNotValidException. A anotação @RestControllerAdvice permite tratar esse erro para os Controllers da aplicação.

O método reúne o nome de cada campo e sua mensagem em um Map. Depois, devolve essas informações no corpo da resposta com o código 400 Bad Request.

ValidacaoExceptionHandler.java

src > main > java > br > com > senai > teste > exception > ValidacaoExceptionHandler.java

```java
package br.com.senai.teste.exception;

import java.util.LinkedHashMap;
import java.util.Map;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class ValidacaoExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> tratarValidacao(
            MethodArgumentNotValidException erro) {

        Map<String, String> mensagens = new LinkedHashMap<>();

        erro.getBindingResult().getFieldErrors().forEach(campo ->
            mensagens.put(campo.getField(),
                          campo.getDefaultMessage())
        );

        return ResponseEntity.badRequest().body(mensagens);
    }
}
```

Ln 27, Col 2 Spaces: 4 UTF-8 CRLF

Mensagem de validação

Atenção



Nesta versão simples, cada campo terá uma mensagem na resposta. Se o mesmo campo descumprir mais de uma regra, o Map conservará apenas uma delas. Isso é suficiente para nosso primeiro tratamento de erros de validação.

Testando as mensagens de erro

Identificando o que precisa ser corrigido no JSON

Reinicie a aplicação para aplicar as alterações.

No Postman, faça duas requisições POST para:

- http://localhost:8080/alunos

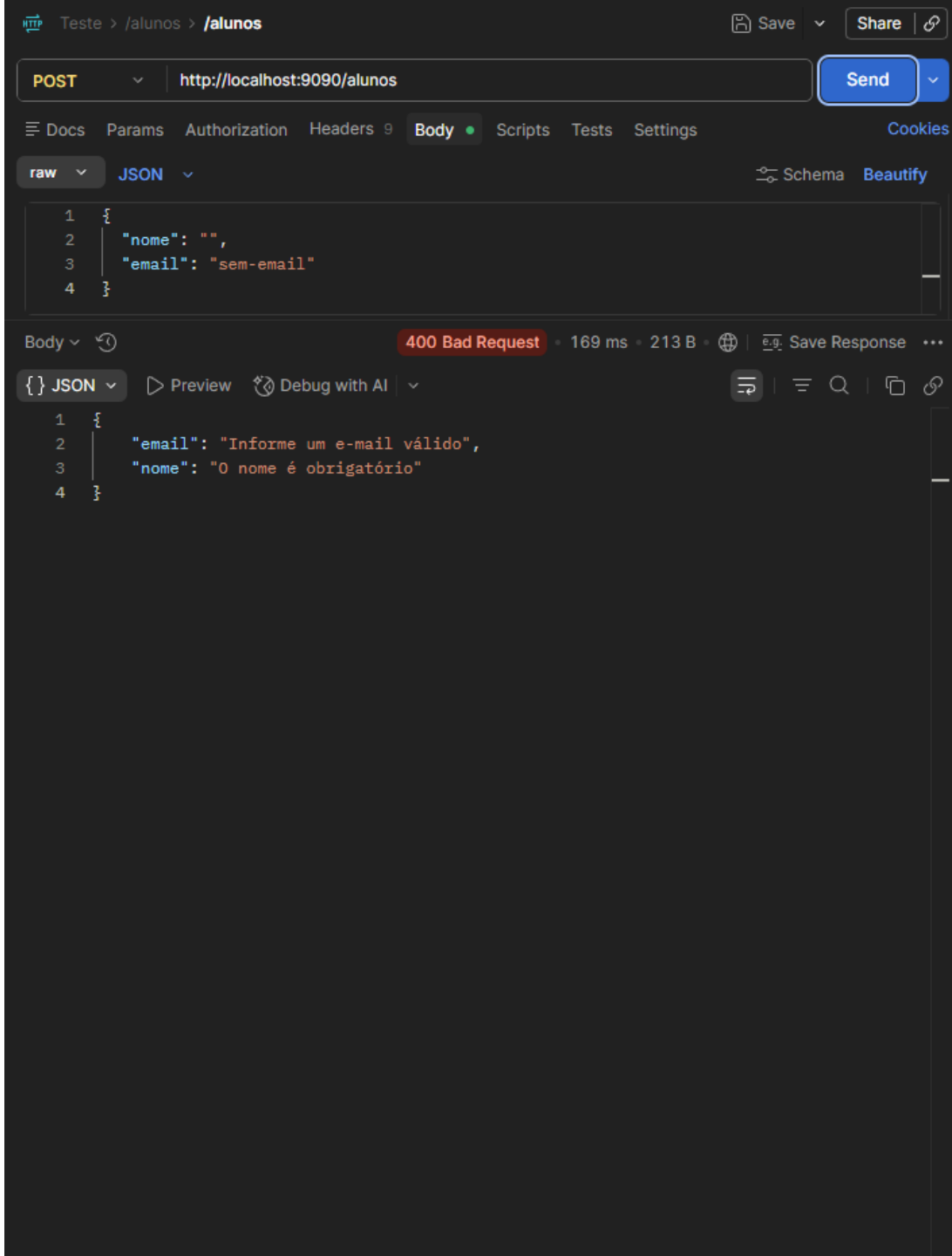
Selecione **Body → raw → JSON e envie**:

-
```
{
  "nome": "",
  "email": "sem-email"
}
```

Resultado esperado: 400 Bad Request.

Cada chave da resposta identifica um campo do JSON enviado. O texto ao lado informa qual regra não foi atendida.

Nesse teste, o Controller não chama o Service e nenhum aluno é cadastrado. Corrija os valores e envie a requisição novamente para verificar o retorno 201 Created.



Validação de Livros

Protegendo o cadastro de livros

No arquivo Livro.java:

- adicione as importações
- acrescente as anotações aos atributos existentes

No LivroController.java, adicione @Valid antes de @RequestBody nos métodos de cadastro e atualização, seguindo o exemplo do AlunoController.

@Min(1) define o menor valor aceito para anoPublicacao. Como o tratamento de erros foi criado com @RestControllerAdvice, ele também devolverá mensagens de validação para as requisições de livros.

Livro.java

```java
package br.com.senai.teste.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;

@Entity
@Table(name = "livro")
public class Livro {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @NotBlank(message = "O título é obrigatório")
    private String titulo;

    @NotBlank(message = "O autor é obrigatório")
    private String autor;

    @Min(value = 1, message = "O ano deve ser maior que zero")
    private int anoPublicacao;

    public Livro() {
    }

    public Livro(String titulo, String autor, int anoPublicacao) {
        this.titulo = titulo;
        this.autor = autor;
        this.anoPublicacao = anoPublicacao;
    }

    public String getTitulo() {
        return titulo;
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public String getAutor() {
        return autor;
    }

    public void setAutor(String autor) {
        this.autor = autor;
    }

    public int getAnoPublicacao() {
        return anoPublicacao;
    }

    public void setAnoPublicacao(int anoPublicacao) {
        this.anoPublicacao = anoPublicacao;
    }
}
```

LivroController.java

```java
package br.com.senai.teste.controller;

import java.util.List;
import java.util.Optional;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import br.com.senai.teste.model.Livro;
import br.com.senai.teste.service.LivroService;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/livros")
public class LivroController {

    private final LivroService livroService;

    public LivroController(LivroService livroService) {
        this.livroService = livroService;
    }

    @PostMapping
    public ResponseEntity<Livro> cadastrar(@Valid @RequestBody Livro livro) {
        Livro livroCadastrado = livroService.cadastrar(livro);

        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(livroCadastrado);
    }

    @GetMapping
    public ResponseEntity<List<Livro>> listar() {…

    @GetMapping("/{id}")
    public ResponseEntity<Livro> buscarPorId(@PathVariable Integer id) {…

    @PutMapping("/{id}")
    public ResponseEntity<Livro> atualizar(…

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> excluir(@PathVariable Integer id) {…
}
```

Validação de livros no Postman

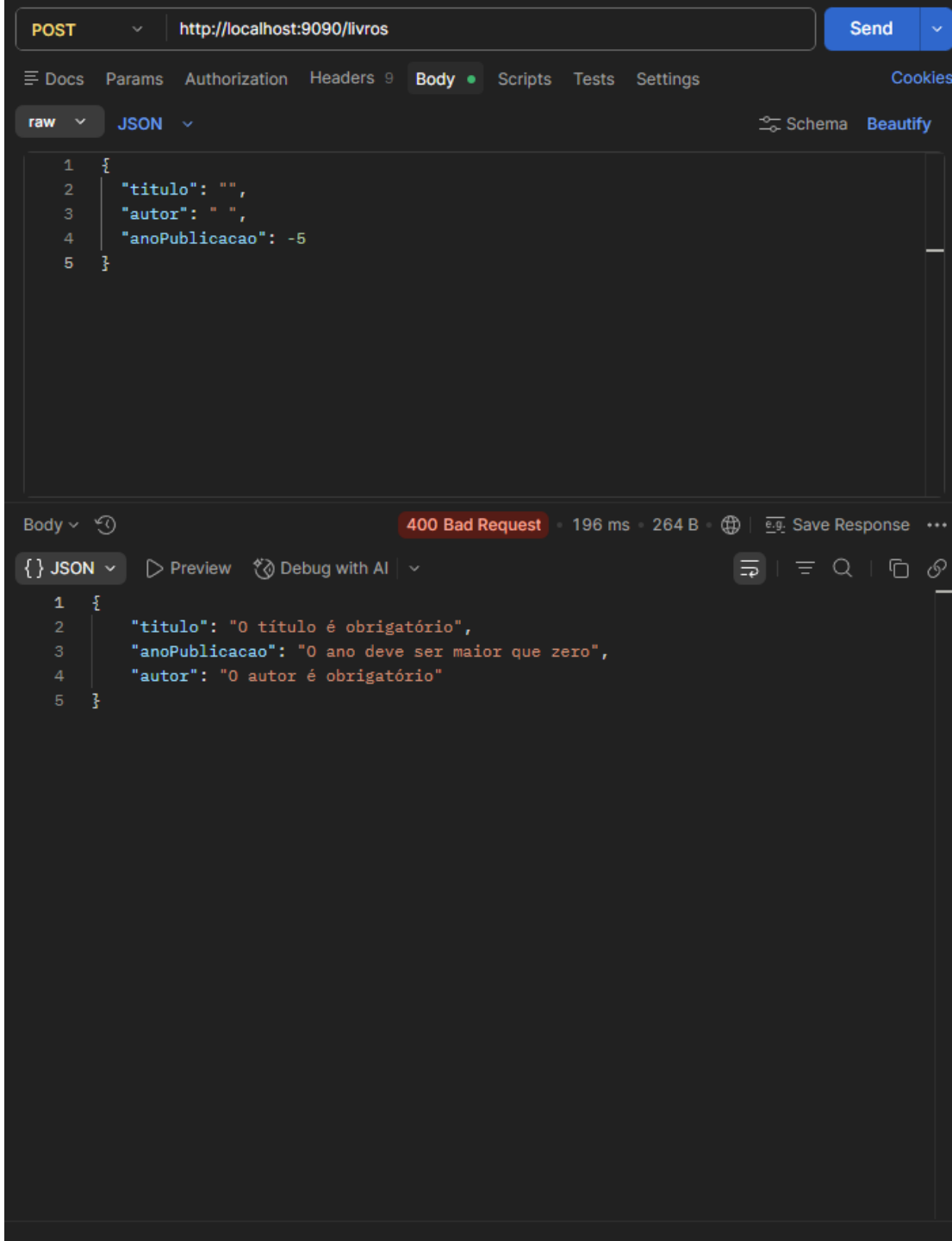
Conferindo o cadastro e a atualização

Com as regras definidas em Livro.java e @Valid aplicado no Controller, vamos enviar dados inválidos pelo Postman. Observe o código HTTP, as mensagens devolvidas e o conteúdo da tabela livro após cada teste.

- Cadastro inválido. Selecione **Body → raw → JSON e envie:**
-

```
{
  "titulo": "",
  "autor": " ",
  "anoPublicacao": -5
}
```

Resultado esperado: 400 Bad Request.



Firjan SENAI